UROP PROJECT

Image Quality Assessment Using Deep Learning

Progress Report 1: Problem Study, Dataset and Model Design

Report	Progress Report 1
Submitted to	Dr. Ratnadeep Dey
Submitted by	
Registration No.	
Date of submission	

1. Introduction

Image Quality Assessment (IQA) is the task of estimating how good or poor an image appears to a human observer. Quality can be reduced by blur, noise, compression, poor exposure, colour distortion, resizing and many other effects introduced while an image is captured or processed. An automatic IQA system is useful whenever large numbers of images must be screened or ranked without asking a person to rate each one, for example when filtering uploads on a photo platform or checking the output of a camera pipeline.

IQA methods are commonly grouped into full-reference, reduced-reference and no-reference methods. Full-reference methods compare a distorted image with an undistorted original, and reduced-reference methods use partial information about that original. In many practical situations the original image simply does not exist, so this project focuses on No-Reference Image Quality Assessment (NR-IQA), in which the model receives only the image whose quality must be predicted. This is the same problem addressed by the reference paper for this project [1].

The project builds a lightweight deep-learning NR-IQA model that uses MobileNetV2 [2] as a pretrained feature extractor and a small regression head that predicts a continuous perceptual quality score. Transfer learning allows the model to reuse visual features learned on ImageNet [3] instead of training a large network from scratch. The model is trained on the KonIQ-10k dataset [4], which contains real-world photographs, each labelled with a Mean Opinion Score (MOS) collected from human observers.

The finished system accepts an image it has not seen before, preprocesses it, predicts a quality score and presents that score in an understandable form. For the demonstration interface, the continuous prediction is additionally mapped to simple descriptive levels (Poor, Average and Good). The numerical prediction remains the primary output; the descriptive level is only a presentation layer on top of it.

This report covers the groundwork required before full training. Section 2 states the objectives, Section 3 presents the background study, Section 4 describes the software environment and the preparation of the dataset, Section 5 sets out the model design, training strategy and inference process, Section 6 discusses the main observations and expected outcomes, and Section 7 concludes the report.

2. Objectives

The project has the following objectives.

1. To develop a no-reference image quality assessment system using deep learning, so that image quality can be estimated without an original reference image.
2. To prepare the KonIQ-10k dataset and its MOS labels for supervised regression.
3. To use an ImageNet-pretrained MobileNetV2 network as a lightweight feature extractor and attach a regression head that predicts the quality score.
4. To train and validate the model in TensorFlow/Keras on Google Colab and save the best model for later inference.
5. To evaluate the predictions using Mean Absolute Error (MAE) together with the Spearman Rank Order Correlation Coefficient (SROCC) and the Pearson Linear Correlation Coefficient (PLCC).
6. To provide a simple demonstration in which an uploaded image receives a numerical quality score and an easy-to-understand quality level.

The present report addresses the first part of this work: understanding NR-IQA, selecting the dataset and model, fixing the preprocessing and evaluation approach, and defining an architecture that can be built and trained with the available resources.

3. Background Study

3.1 Image quality assessment and NR-IQA

Many objective IQA measures depend on a high-quality reference image. This works well in controlled settings such as compression testing, but it is restrictive for photographs taken from cameras, social media or large image collections, where the pristine source is usually unavailable. NR-IQA removes this dependence by learning, directly from the test image, the visual patterns that correlate with perceived quality. Earlier NR-IQA methods relied on hand-crafted features; BRISQUE, for example, measures how far the local statistics of an image depart from those of natural, undistorted scenes and maps these measurements to a quality score [5]. Learned methods replace the hand-crafted features with features extracted by a deep network, which generally cope better with the mixed and irregular distortions found in real photographs.

The reference paper for this project, Sang et al. [1], studies the same NR-IQA problem at research scale. Its pipeline uses generated reference images, synthetic degradations produced with a cold diffusion model, self-supervised pre-training, a Vision Transformer backbone and custom attention modules. The present project does not reproduce those research contributions. It keeps the same core problem, predicting perceptual quality without a reference image, and implements it with a lightweight CNN transfer-learning pipeline that is practical on Google Colab.

3.2 Deep learning for image quality

A convolutional neural network learns hierarchical visual representations. Early layers respond to local patterns such as edges and textures, while deeper layers encode larger and more complex structures. For IQA, these representations carry information related to sharpness, loss of texture, noise, contrast and other changes that influence how good an image looks. Instead of

predicting an object category, the final part of the network is changed so that it outputs a single continuous quality value.

This makes the project a regression problem rather than an image-classification problem. The target is not a class such as ‘cat’ or ‘car’ but the human quality score attached to each training image, and a prediction that lies close to that score indicates a useful model.

3.3 MobileNetV2 and transfer learning

MobileNetV2 [2] is selected as the backbone because it offers a good balance between representational power and computational cost. It is built from inverted residual blocks with linear bottlenecks and uses depthwise separable convolutions, which reduce computation considerably compared with standard convolutions. For a 224×224 input, the convolutional part of the network produces a 7×7 feature map with 1280 channels and contains about 2.26 million parameters, which makes it small enough to train comfortably on a free Colab GPU. For a project of this scope, this is a far more practical choice than a large Vision Transformer with custom attention modules.

The backbone is initialized with ImageNet-pretrained weights and its original classification layer is removed. The convolutional layers turn the input image into high-level feature maps, Global Average Pooling reduces those maps to a compact feature vector, and a small fully connected regression head predicts one quality score. This is transfer learning: the model reuses general visual knowledge and learns only the quality-specific mapping from the IQA dataset.

3.4 Regression loss and evaluation metrics

Let y_i be the ground-truth MOS of image i, $\hat{y}_i$ the predicted score and N the number of images. Mean Squared Error (MSE) is used as the training loss because it penalizes large errors strongly and suits continuous regression:

$$\mathrm{MSE} = (1/N)\ \Sigma_i\ (\hat{y}_i - y_i)^2 \tag{1}$$

For reporting, the Mean Absolute Error gives the average size of a prediction error in the same units as the MOS:

$$\mathrm{MAE} = (1/N)\ \Sigma_i\ |\hat{y}_i - y_i| \tag{2}$$

Error measures alone are not enough for IQA. A useful quality model must also order images the way people do, and its predictions should rise and fall linearly with the human scores. The IQA literature therefore reports two correlation coefficients. SROCC measures agreement in ranking, where d_i is the difference between the rank of image i in the predictions and its rank in the ground truth:

$$\mathrm{SROCC} = 1 - 6\ \Sigma_i\ d_i^2\ /\ (\ N\ (N^2 - 1)\ ) \tag{3}$$

PLCC measures linear agreement, where $\mu_{\hat{y}}$ and μ_y are the means of the predicted and true scores:

$$\mathrm{PLCC} = \Sigma_i\ (\hat{y}_i - \mu_{\hat{y}})(y_i - \mu_y)\ /\ \sqrt{[\ \Sigma_i\ (\hat{y}_i - \mu_{\hat{y}})^2 \cdot \Sigma_i\ (y_i - \mu_y)^2\ ]} \tag{4}$$

Equation (3) holds when there are no tied ranks; the SciPy implementation used in the project handles ties automatically. Table 1 summarizes how each metric is read.

Table 1. Evaluation metrics

Metric	Meaning	Better when
MAE	Average absolute difference between predicted and human scores	Lower
SROCC	Agreement between the predicted and the human ranking of images	Closer to +1
PLCC	Linear correlation between predicted and human scores	Closer to +1

4. Dataset and Data Preparation

Figure 1 gives an overview of how the downloaded KonIQ-10k files are turned into batches that the network can use. The subsections below explain each step and the reasoning behind it.

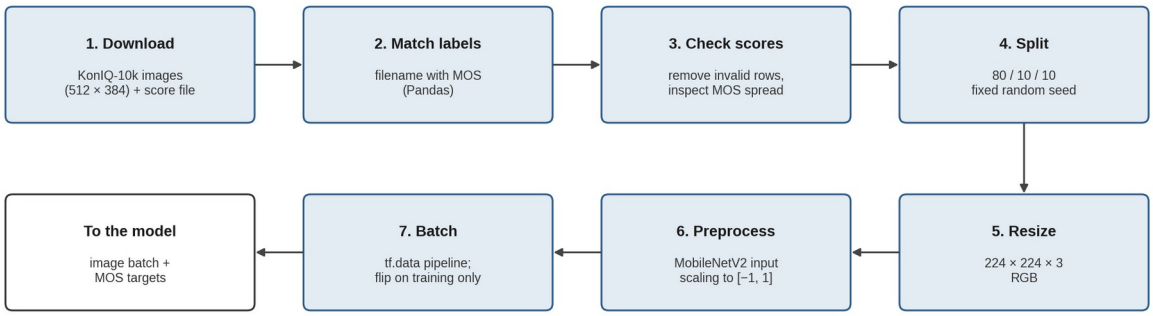


Figure 1. Data preparation pipeline for KonIQ-10k

4.1 Software environment

The implementation is developed in Python on a Google Colab GPU runtime. TensorFlow [6] with its Keras API is used for model definition, training and inference, and supporting libraries handle image loading, numerical work, plotting and metric calculation. Table 2 lists the tools and the role of each.

Table 2. Software used in the project

Library / Tool	Purpose
Google Colab	Notebook environment with a free GPU runtime for training
Python	Programming language for the whole project
TensorFlow / Keras	MobileNetV2, model definition, training and inference
NumPy	Numerical and array operations
Pandas	Reading and managing image file names and MOS labels
Pillow / OpenCV	Image loading and basic image measurements
Matplotlib	Dataset samples, training curves and prediction plots
SciPy	SROCC and PLCC calculation
Streamlit	Upload-and-predict interface for the demonstration

4.2 KonIQ-10k dataset

KonIQ-10k [4] was chosen because it was built specifically for authentic image quality assessment. Its 10,073 images were sampled from the public YFCC100M collection of Flickr photographs, so the distortions in them (noise, compression artefacts, motion blur, poor exposure and so on) arise naturally rather than being added artificially. Each image was rated by crowd workers, giving about 1.2 million ratings in total, and duplicates were removed while the dataset was built. The ratings are averaged into a Mean Opinion Score for each image, and this MOS is the supervised target of the regression model.

Only the image and its quality score are needed. No reference image is supplied at any point, which matches the NR-IQA setting exactly: an image goes in and a perceptual quality score comes out. The images are distributed at 1024×768 pixels and also in a reduced 512×384 version; the smaller version is used in this project to save storage and loading time, since every image is resized further before it enters the network. Table 3 summarizes the dataset and its role in the system.

Table 3. KonIQ-10k and its role in the proposed system

Property	Value / project use
Images	10,073 real-world photographs
Source	Flickr photographs from the YFCC100M collection
Distortion type	Authentic (captured in the wild), not synthetic
Resolution	1024 × 768; a 512 × 384 version is used here
Ratings	About 1.2 million crowd-sourced ratings
Rating scale	5-point scale, averaged into one MOS per image
Task	No-reference image quality assessment
Input / target	RGB image / human MOS
Learning type	Supervised regression
Reference image at inference	Not required

4.3 Data partitioning

The dataset is divided into training, validation and test subsets in the ratio 80:10:10, using a fixed random seed so that the same partition is reproduced in every run. Because KonIQ-10k contains distinct photographs with duplicates already removed, the images can be split individually without the risk of near-identical pictures appearing on both sides of the split. The training set is used to learn the model parameters, the validation set guides decisions such as checkpoint selection and learning-rate adjustment, and the test set is kept aside until the final evaluation. The resulting sizes are shown in Table 4.

Table 4. Division of KonIQ-10k

Subset	Share	Images
Training	80%	8,058
Validation	10%	1,007
Test	10%	1,008
Total	**100%**	**10,073**

Before training, the MOS distribution of each subset will be compared with that of the full dataset, so that all three contain a reasonable spread of low-, medium- and high-quality images. The counts above follow from the chosen ratio and will be confirmed against the metadata file once the download has been verified in the notebook.

4.4 Preprocessing

Each image passes through a small, consistent preprocessing pipeline before it enters MobileNetV2.

1. The image is read in RGB format and paired with its MOS from the metadata file. OpenCV reads images in BGR order, so any image loaded with OpenCV is converted to RGB first.
2. The image is resized to 224×224 pixels, the standard input size of the pretrained backbone, which keeps training light. This changes the 4:3 aspect ratio and removes some fine detail, both of which can affect perceived quality. The trade-off is accepted for the baseline model, and training at the native 384×512 resolution, which MobileNetV2's convolutional layers also accept, is kept as a possible extension if time and compute allow.
3. The image is converted to a floating-point tensor and passed through the MobileNetV2 preprocessing function, which scales pixel values to the range [−1, 1] expected by the pretrained weights. The same function is used at inference time.
4. The MOS is kept as a continuous regression target. It may be rescaled to the range 0 to 1 for more stable optimization, in which case predictions are converted back to the original scale before they are reported.
5. Augmentation is limited to random horizontal flipping of training images. Validation and test images are not augmented.

The last point matters more in IQA than in classification. For a classifier, heavy augmentation builds useful invariance, but in quality assessment added blur, noise, compression or strong colour shifts change the very property the model is supposed to predict, so the image would no longer match its MOS. A horizontal flip changes the content layout without changing quality, which is why it is the only augmentation used.

5. Model Design

The proposed model is deliberately compact. MobileNetV2 performs feature extraction, and a small regression head turns the learned representation into one quality score, as shown in Figure 2. No segmentation decoder, diffusion model, Vision Transformer or custom attention module is needed for this implementation.

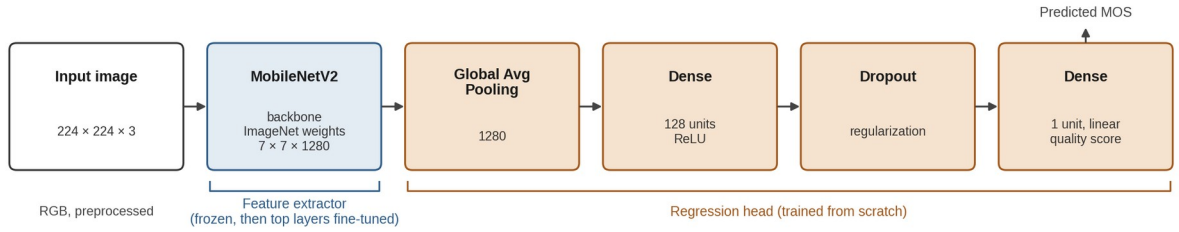


Figure 2. Proposed MobileNetV2 regression architecture

5.1 Feature extractor

The input is an RGB image of size 224×224. MobileNetV2 is loaded without its ImageNet classification layer, so its output is the 7×7×1280 feature map from the last convolutional block. At the start of training all backbone layers are frozen, so that the new regression head can learn without immediately altering the pretrained filters. Once the head is stable, a limited number of upper MobileNetV2 layers are unfrozen and fine-tuned with a smaller learning rate.

5.2 Regression head

The final feature map is reduced by Global Average Pooling to a vector of 1280 values. A dense layer with ReLU activation, starting at 128 units, learns a quality-specific representation, and dropout is applied after it to reduce overfitting. The output layer is a single neuron with linear activation that produces the predicted quality score. Table 5 lists each stage with its output shape and number of parameters.

Table 5. Layers of the proposed model

Stage	Operation	Output shape	Parameters
Input	224 × 224 × 3 RGB image	224 × 224 × 3	0
Backbone	ImageNet-pretrained MobileNetV2, no top	7 × 7 × 1280	≈ 2.26 million
Pooling	Global Average Pooling	1280	0
Dense	128 units, ReLU	128	163,968
Dropout	Regularization during training	128	0
Output	1 unit, linear	1	129

The head adds only about 164 thousand parameters to the backbone, so almost all of the model's capacity comes from the pretrained features. This is one reason the model can be trained on around eight thousand images without severe overfitting.

5.3 Training strategy

Training is carried out in two phases, summarized in Table 6. In the first phase the pretrained backbone is frozen and only the regression head is trained. In the second phase the upper blocks of the backbone are unfrozen and fine-tuned together with the head using a much smaller learning rate. During fine-tuning the batch-normalization layers of the backbone are kept in inference mode, because updating their statistics on small batches can undo the pretrained behaviour. Adam [7] is used as the optimizer and MSE as the loss, and the checkpoint with the lowest validation loss is saved.

Table 6. Two-phase training plan

Phase	Trainable layers	Starting learning rate	Purpose
1	Regression head only	10^{-3}	Learn the quality mapping on fixed features
2	Head and upper MobileNetV2 blocks	10^{-5}	Adapt the features to quality cues

The batch size and number of epochs are chosen according to the Colab GPU memory and the validation behaviour rather than fixed in advance, and the learning rates above are starting values to be adjusted on the validation set. Early stopping ends a phase when the validation loss stops improving. This keeps training efficient while still giving a technically meaningful procedure.

5.4 Inference and quality-level mapping

After training, the saved model is loaded for inference, as shown in Figure 3. A user-supplied image undergoes exactly the same resizing and MobileNetV2 preprocessing used during training, and the network outputs one continuous score. For the demonstration interface, the score is also converted into Poor, Average or Good using thresholds taken from the MOS distribution of the training set (for example, its tertiles) rather than arbitrary fixed values. This conversion is only a readable interpretation of the regression output; there is no separate classification model.

The interface can also display simple image measurements computed with OpenCV: sharpness as the variance of the Laplacian, brightness as the mean intensity and contrast as the standard deviation of intensity. These are presented clearly as auxiliary diagnostics. They help a viewer understand the image, but they are not explanations produced by the deep-learning model, which does not use them.

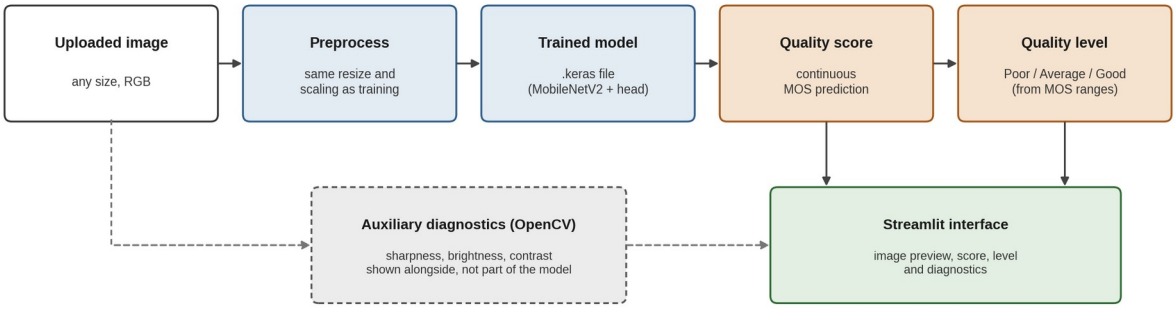


Figure 3. Inference flow in the demonstration application

6. Observations and Expected Outcomes

Several design decisions are fixed at this stage. The task remains strictly NR-IQA, so the model receives only the image under assessment at inference time. The target is a continuous human quality score, which makes regression the appropriate formulation rather than segmentation or ordinary classification. Transfer learning makes the project practical, because a capable visual backbone can be adapted without the cost of training a deep CNN from random initialization.

The choice of MobileNetV2 also controls the scope of the work. The reference paper [1] demonstrates a research-level solution with a Vision Transformer backbone, generated pre-training data and attention modules. Reproducing that system would greatly increase the compute, implementation time and experimental effort required. A lightweight pretrained CNN allows the central NR-IQA idea to be demonstrated and evaluated cleanly within the available resources.

The main experimental risk is overfitting, since IQA datasets are far smaller than large classification datasets. Freezing the backbone at first, using dropout, monitoring the validation loss and limiting augmentation are the planned safeguards. A second risk is relying on a single error figure as the evidence of success, which is why the evaluation combines MAE with SROCC and PLCC and will also show example images with their true and predicted scores.

Two properties of the dataset will shape the results. Most KonIQ-10k images have mid-range scores and very few are extremely poor or excellent, so the model is expected to predict extreme scores less accurately and to pull them towards the middle of the range. In addition, because the dataset contains authentic distortions, images with heavy artificial degradation, such as strong synthetic blur, lie outside what the model has learned; the demonstration will therefore use real photographs with naturally occurring quality problems.

The expected demonstration is straightforward: an image is uploaded, preprocessed and scored, its descriptive quality level is shown, and the basic diagnostic measurements are displayed alongside. The application does not train the model; it only loads the saved model and performs inference.

7. Conclusion

This report establishes the foundation of the project on image quality assessment using deep learning. The task is defined as no-reference image quality assessment on the KonIQ-10k dataset with human MOS labels. MobileNetV2 with ImageNet transfer learning is selected as a lightweight feature extractor, followed by a small regression head that predicts one continuous quality score, and the data partitioning, preprocessing and two-phase training strategy have been fixed with the reasons for each choice.

The implementation uses Python, TensorFlow/Keras and Google Colab. The model is trained as a regression system with MSE and evaluated with MAE, SROCC and PLCC, and a Streamlit interface presents the predicted score together with a Poor, Average or Good level. The project stays aligned with the NR-IQA direction of the reference paper while deliberately leaving out its research-scale generative, diffusion and attention components, which keeps it technically sound and feasible on free computing resources. The work now moves on to verifying the downloaded dataset, implementing the data pipeline and model in Colab, and running the first training experiment.

References

[1] Q. Sang, Q. Li, L. Liu, Z. Deng, X. Wu, and A. C. Bovik, “No-reference image quality assessment leveraging GenAI images,” IEEE Transactions on Image Processing, vol. 34, pp. 6204–6214, 2025.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510–4520.

[3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248–255.

[4] V. Hosu, H. Lin, T. Sziranyi, and D. Saupe, “KonIQ-10k: An ecologically valid database for deep learning of blind image quality assessment,” IEEE Transactions on Image Processing, vol. 29, pp. 4041–4056, 2020.

[5] A. Mittal, A. K. Moorthy, and A. C. Bovik, “No-reference image quality assessment in the spatial domain,” IEEE Transactions on Image Processing, vol. 21, no. 12, pp. 4695–4708, 2012.

[6] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: https://www.tensorflow.org

[7] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. Int. Conf. Learning Representations (ICLR), 2015.